

Generate and Measure 60 Hz V-Sync in X68000 31 kHz Mode

From the MS.X implementation to a C experiment and emulator improvements

Vertical timing basics · CRTC R04–R07 · V-DISP measurement · Emulator comparison

MS.X treated the 60 Hz versus 55 Hz gap as its first display obstacle

“The MSX runs at 60 Hz in NTSC mode. The normal display mode of the X68000, however, runs at approximately 55 Hz. At this rate, the vertical-sync timing does not match.” [1, p.35; translated]

“Many games run in reference to the moment the screen is rewritten (vertical sync), so when this timing differs, everything goes wrong.” [1, p.35; translated]

- 1 MSX NTSC timing is about 60 Hz
- 2 Standard X68000 31 kHz timing is about 55.5 Hz
- 3 MS.X changes the CRTC timing directly

The C version measures V-DISP to confirm that the same timing change really reaches 60 Hz.

Faster hardware does not automatically reproduce the same timing behavior

Higher performance and faithful behavior are different requirements.

1

Processing capacity

Extra CPU and graphics capacity does not remove a synchronization mismatch.

2

Timing compatibility

The vertical synchronization observed by software must run at the intended cadence.

3

Measurement

Verify the generated signal in real time instead of trusting register values alone.

FREQUENCY MISMATCH

MSX NTSC : about 60 Hz
X68000 31 kHz : about 55.5 Hz

performance != timing compatibility

MS.X Development Story, PDF pp.9 and 37

The C experiment turns that lesson into a 525-line setting followed by 600 measured V-DISP intervals.

Approach 60 Hz without changing the horizontal line timing

Use the MS.X implementation to build a 525-line frame, then measure it with a C program.

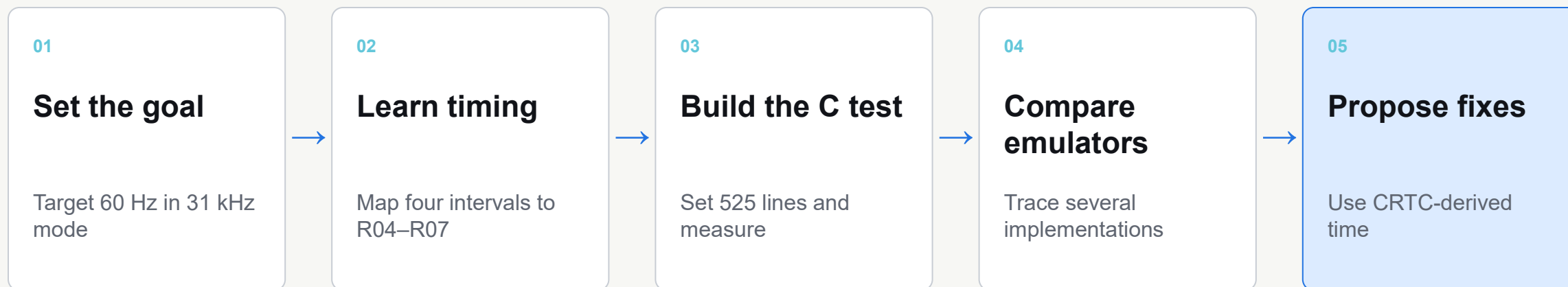
The standard 31 kHz-class mode runs at about 55.5 Hz. Keeping the horizontal rate near 31.5 kHz and changing one frame to 525 lines targets about 60 Hz.

- 1 First understand the four signal intervals and R04–R07
- 2 Then build a restorable C experiment from the MS.X settings
- 3 Finally use measurements to identify emulator problems and improvements

Measure whether the configured vertical sync is really 60 Hz.

Follow five stages from MS.X to emulator improvements

Understand timing and register meaning before reading values and code.



The first half explains generation and measurement; the second examines how emulators reproduce them.

One frame contains sync, blanking, and visible intervals

Back porch: after VSYNC, before display.

Front porch: after display, before the next VSYNC.

1**R04**

The final raster of a frame; $R04 + 1$ is the total line count.

2**R05 and R06**

R05 ends VSYNC; R06 ends the back porch.

3**R07**

Ends the visible interval; the front porch starts next.

VERTICAL TIMING ORDER

VSYNC

- > back porch
- > visible area
- > front porch
- > next VSYNC

Source: XEiJ CRTC.java (R04-R07 register definitions)

R04–R07 are end positions of intervals, not interval lengths.

Confirm the R04–R07 addresses in the MS.X source

The C version accesses only the vertical timing registers needed for 60 Hz.

1

R04

\$E80008 sets the end position of the vertical frame.

2

R05 and R06

\$E8000A / \$E8000C set sync and display boundaries.

3

R07

\$E8000E sets the end of the visible interval.

SRC/VDP/MS_VDP.H

```
#define CRTR_04 (*(volatile uint16_t*)0xe80008)
#define CRTR_05 (*(volatile uint16_t*)0xe8000a)
#define CRTR_06 (*(volatile uint16_t*)0xe8000c)
#define CRTR_07 (*(volatile uint16_t*)0xe8000e)
```

ms_vdp.h:28-37

The MS.X definitions identify the four addresses written by the C test.

525 lines at 31.5 kHz produces 60 Hz

$$31,500 \text{ lines/s} \div 525 \text{ lines/frame} = 60.00 \text{ frames/s}$$

HORIZONTAL RATE

31.5 kHz

Keep the line duration unchanged

FRAME TOTAL

525 lines

Sum of the four intervals

VERTICAL RATE

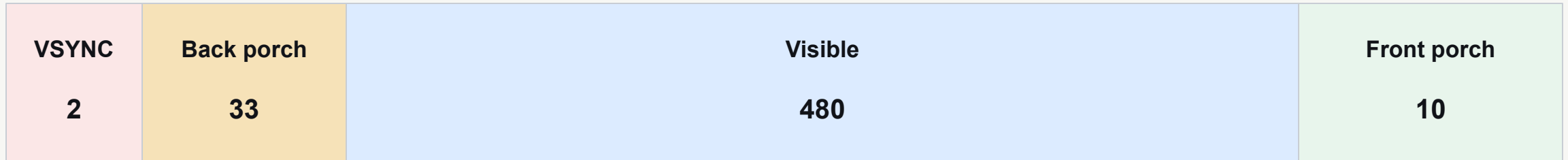
60.00 Hz

Frames generated each second

Vertical frequency is determined by horizontal line duration and total lines per frame.

Accumulate four interval lengths to derive R04–R07

Place 2 VSYNC, 33 back-porch, 480 visible, and 10 front-porch lines in order.



$$2 + 33 + 480 + 10 = 525 \text{ lines}$$

R05=1 → R06=34 → R07=514 → R04=524

Each register stores an interval's final position, so the value is one less than the count.

Write the MS.X-derived settings to R04–R07 in C

垂直CRTC設定

```
R04 = 0x020C; /* 525 - 1 */  
R05 = 0x0001; /* VSYNC end */  
R06 = 0x0022; /* display start */  
R07 = 0x0202; /* display end */
```

MS.X / crt60hz.c

Use the MS.X vertical timing settings to build a 525-line frame in C.

1

Start from a standard mode

IOCS mode 12 provides known 31 kHz-class horizontal timing.

2

Change vertical values only

Write the four previously derived values to R04–R07.

3

Measure after generation

Count V-DISP events and compare them with elapsed time.

MS.X supplies the setting rationale; the C program measures the generated vertical sync.

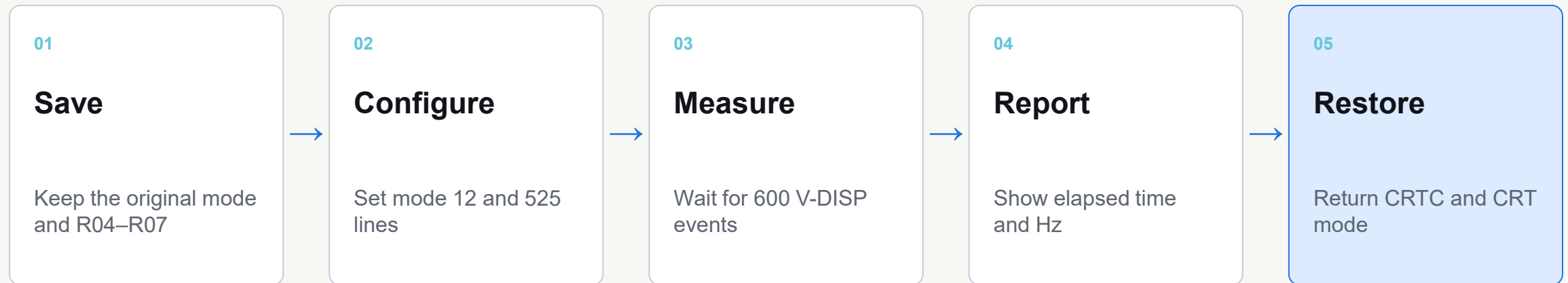
Use MS.X as the reference, then add measurement and safe restoration in C

Item	crtc60hz.c	MS.X
Primary goal	Measure vertical frequency	Provide an MSX display environment
CRTC scope	Change R04–R07 only	Configure the complete mode
Frequency check	Measure 600 V-DISP events	Use a one-second tick
Exit handling	Restore saved values	Restore during MS.X shutdown

Build the C version from MS.X, then measure the frequency of the generated vertical sync.

Save the CRTC settings and screen mode before changing them, then restore them after measuring

Direct CRTC writes are safe only when restoration is part of the same flow.



Normal completion, ESC, and timeout all execute the same restoration path.

Set the standard 31 kHz mode, then change only vertical timing

処理の要点

```
_iocs_crtmod(12);  
  
save_crtc();  
write_crtc(R04, 0x020c);  
write_crtc(R05, 0x0001);  
write_crtc(R06, 0x0022);  
write_crtc(R07, 0x0202);
```

crtc60hz.c

Keep horizontal timing known and limit direct register writes.

1

Start from a standard mode

IOCS creates a known 31 kHz-class state.

2

Save first

Read R04–R07 before direct writes.

3

Limit the scope

Do not modify unrelated timing registers.

Separate the risky hardware work into save, configure, and restore operations.

Watch GPIP bit 4 and wait for the next V-DISP rise

Count one new display start regardless of the current signal level.

1**GPIP bit 4**

General Purpose Input/Output Port.
\$10 reflects V-DISP (0: blanking, 1: display); \$40 is CIRQ.

2**Wait for Low first**

Do not count the current High interval twice.

3**Use a timeout**

Return to restoration if the signal stops.

WAIT_VDISP()

```
#define GPIP_VDISP 0x10
```

```
while ((read_gpip() & GPIP_VDISP) != 0) { }  
while ((read_gpip() & GPIP_VDISP) == 0) { }
```

crtc60hz.c

One wait_vdisp() call counts one following V-DISP start.

Read the end time immediately after V-DISP number 600

測定区間

```
wait_vdisp();
start = _iocs_ontime();

for (frame = 1; frame <= 600; ++frame) {
    wait_vdisp();
    if (frame == 600) end = _iocs_ontime();
}
```

crtc60hz.c

Align both timestamps to V-DISP boundaries.

1

Start

Read start immediately after the first boundary.

2

600 intervals

Wait for 600 additional V-DISP events.

3

End

Read end before drawing the next frame.

$$\text{Hz} \times 100 = 600 \times 10000 \div \text{elapsed time in } 1/100 \text{ s}$$

Reduce drawing work and restore the display on every exit

The flow line is a visual aid, not part of the frequency measurement.

Update the line at about 30 Hz and erase it before exit.
Restore cursor state, R04–R07, and the IOCS CRT mode through one shared cleanup path.

- 1 Reduce drawing so it does not disturb measurement
- 2 Erase the flow line and graphics before restoring the mode
- 3 Use the same cleanup for completion, ESC, and timeout

```
X68000 60 Hz V-DISP measurement
Measuring 600 frames...
Progress   : 360 / 600
FPS : 60.00

ESC : abort
```

Normal operation: V-DISP counting with an FPS display and one-pixel flow line

Real hardware, XEiJ, and XM6 TypeG measure about 60 Hz

The same program produces different results across emulator implementations.

Real X68000

about 60 Hz

525-line timing and restoration confirmed

XEiJ / XM6 TypeG

about 60 Hz

V-DISP rate and clean exit confirmed

WebX68k / MPX68k

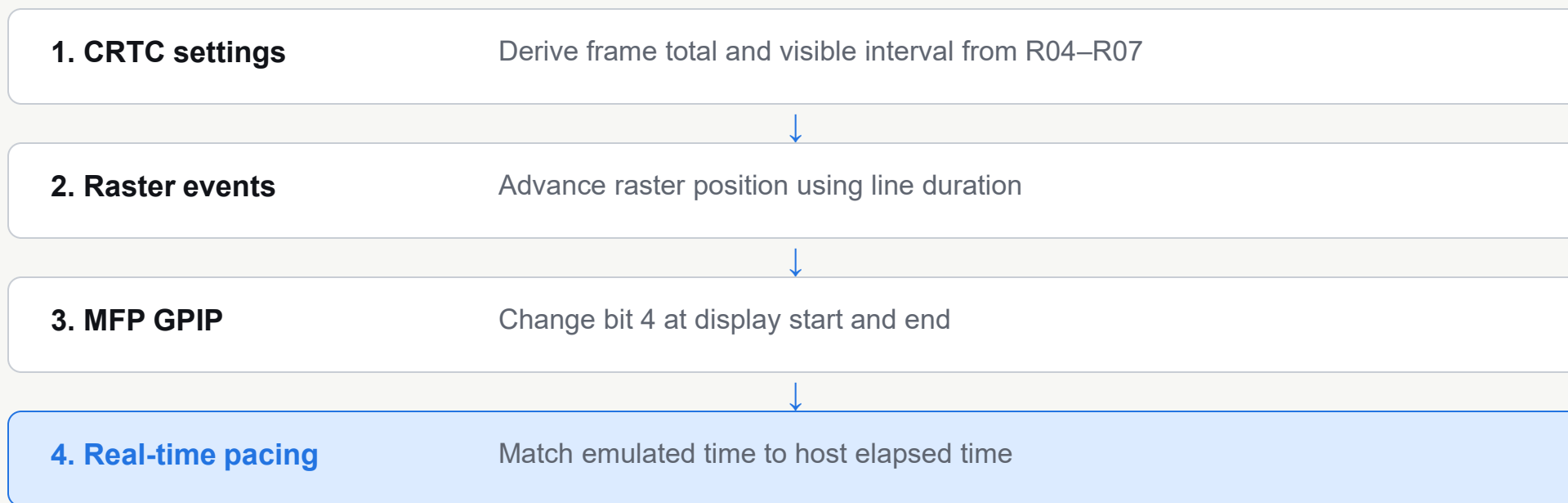
mismatch

Their timing path needs investigation

The likely difference is how each emulator converts CRTC values into elapsed time.

Connect CRTC values to real time through four distinct stages

A correct GPIIP bit value alone does not guarantee the correct frequency.



Use XEiJ and XM6 TypeG to derive the implementation requirements.

XEiJ changes V-DISP at CRTC-derived raster times

XEIJ/CRTC.JAVA

```
crtVDispStart = crtR06VBackEndCurr + 1;  
crtVIdleStart = crtR07VDispEndCurr + 1;  
  
MC68901.mfpVdispRise();  
  
TickerQueue.tkqAdd(  
    crtTicker = NormalDrawDispSync,  
    crtClock += crtHFrontLength);
```

CRTC.java

Use CRTC event time, not host presentation count.

1

Derive boundaries

R06+1 and R07+1 become event positions.

2

Notify the MFP

Call mfpVdispRise() at display start.

3

Accumulate time

Add each horizontal interval to crtClock.

Signal generation is independent from host image presentation.

Trace CRTC, MFP, and VM timing separately in XM6 TypeG

Connect the measured 60 Hz result to the signal-generation path.

1 CRTC

Confirm that R04–R07 update vertical timing.

2 MFP

Trace the V-DISP event to GPIIP bit 4.

3 VM time

Trace how CPU and video events follow real time.

実装の対応関係

```
CRTC register write
-> vertical timing update
-> V-DISP event
-> MFP GPIIP update
-> VM time synchronization
```

Source(VisualStudio2019).zipから引用

Source: Source(VisualStudio2019).zipから引用

Working implementations separate video signals from host drawing

Required behavior	XEiJ	XM6 TypeG	Improvement requirement
Derive frame timing	Recomputes raster boundaries	Managed by CRTC code	Do not use fixed host fps
Generate V-DISP	Calls an MFP event	Updates the MFP state	Generate GPIIP bit 4 independently
Pace real time	Accumulates event time	Advances VM time	Separate from presentation
Apply register writes	Restarts CRTC processing	Updates timing	Recompute after R04 writes

Skipping a host draw must not skip a CRTC event.

WebX68k must separate host refresh from CRTC frame time

```
X68000 60 Hz V-DISP measurement
```

```
Measuring 600 frames...  
Progress   : 540 / 600  
FPS : 57.14
```

```
ESC : abort
```

Observed in WebX68k: FPS 57.14 during measurement

問題が生じる構造

```
requestAnimationFrame()  
  -> run one guest frame  
  -> update V-DISP  
  -> present canvas
```

WebX68k / PX68k系実装の調査

A browser's roughly 60 Hz presentation loop cannot represent arbitrary CRTC timing.

1

Fixed update

R04 writes do not change one host callback interval.

2

Signal tied to drawing

V-DISP follows Canvas presentation count.

3

Dropped draws

A skipped draw may also skip a signal event.

Separate WebX68k CRTC events from browser refresh

Generate signals in emulated time and use the browser callback only to present images.

1

Recompute timing

Derive frame period from R04 and line timing.

2

Schedule GPIIP

Change bit 4 at R06/R07 boundaries.

3

Separate drawing

Drop presentation work, not CRTC events.

改善案 (擬似コード)

```
while (emuTime < hostTargetTime) {  
    runCpuUntil(nextCrtcEvent);  
    updateRasterAndGpip();  
}  
  
if (frameImageReady) {  
    presentOnRequestAnimationFrame();  
}
```

改善案

The standard 55.5 Hz mode and the 525-line 60 Hz mode must both measure correctly.

MPX68k's fixed 55.45 Hz step hides CRTC timing changes

```
X68000 60 Hz V-DISP measurement
```

```
Measuring 600 frames...  
Progress   : 540 / 600  
FPS : 57.14
```

```
ESC : abort
```

ホスト側スケジューラ

```
emulatorHz = 55.45  
targetFrameTime = 1.0 / 55.45  
  
while accumulator >= targetFrameTime {  
    X68000_Update(clockMHz, 0)  
}
```

MPX68Kホスト側コード

A 60 fps window does not make a guest running at a fixed 55.45 Hz step become 60 Hz.

1

Fixed period

The update interval is independent of R04.

2

Host fps is separate

Changing presentation rate leaves guest time unchanged.

3

CPU MHz is separate

Work per update does not repair vertical timing.

Reference: the same symptom observed in WebX68k; this is not an MPX68K capture.

Expose CRTC-derived frame timing from the MPX68k core

Replace fixed 55.45 Hz pacing with the period calculated from the current CRTC state.

1

Calculate in the core

Return line time × total lines.

2

Pace on the host

Run the core for elapsed real time.

3

Present completed images

Send only completed frames to SpriteKit.

改善案 (インターフェース)

```
typedef struct {
    uint64_t frame_period_ns;
    uint32_t total_lines;
    uint32_t vdisp_start;
    uint32_t vdisp_end;
} crtc_timing_t;

void crtc_get_timing(crtc_timing_t *t);
```

改善案

Changing 10/16 MHz must not change vertical frequency when CRTC timing is unchanged.

Verify both the standard 55.5 Hz mode and the 60 Hz modification

Check	Standard mode	525-line setting
Vertical rate	about 55.5 Hz	about 60.0 Hz
600 V-DISP	about 10.81 s	about 10.00 s
GPIP bit 4	changes every frame	changes every frame
Host 60/120 Hz	measurement unchanged	measurement unchanged
CPU 10/16 MHz	period unchanged	period unchanged

Increasing drawing load must not change the V-DISP period.

Generated and measured 60 Hz in 31 kHz mode, then identified emulator improvements

- 1 Derived the 525-line register values from the four signal intervals and R04–R07.
- 2 Built the C experiment from MS.X and confirmed that 600 V-DISP intervals take about ten seconds.
- 3 Compared emulator implementations and identified the need to generate V-DISP from CRTC-derived time.

Acknowledgements

We gratefully acknowledge the published sources, code, and shared knowledge that made this guide possible.

In preparing this guide, we consulted public documentation and source code from MS.X, XEiJ, XM6 TypeG, PX68k, WebX68k, MPX68K, and elf2x68k. We are deeply grateful to the developers who have made these works available and continue to maintain them.

We also express our sincere appreciation to the X68000 community for preserving and sharing technical documentation, development environments, software, and verification results over many years.

X68000 COMMUNITY

References

Bracketed numbers in the slides refer to the entries below. Online sources were accessed on 18 August 2026.

[1] kunichiko, MSX Development Story: Running MSX on X68000, 30 Years of Making, e-book v1.0, 2026 (in Japanese).

<https://kunichiko.booth.pm/items/8672923>

[2] kunichiko, “X68000_MSX_Simulator,” GitHub repository, commit 55cc131, 2026.

https://github.com/kunichiko/X68000_MSX_Simulator

[3] stdkmd, “XEiJ,” CRTc.java and misc-include-vector.equ, source code.

<https://stdkmd.net/xeij/source/xeij-CRTc.java.htm>

[4] NXP Semiconductors, MC68901 Multi-Function Peripheral User’s Manual.

<https://www.nxp.com/docs/en/reference-manual/MC68901UM.pdf>

[5] XM6 TypeG project, “Source(VisualStudio2019).zip,” source archive.

<http://retropc.net/pi/xm6/index.html>

[6] hissorii et al., “PX68k / px68k-libretro,” GitHub repositories, commits 74ddbb3 / bb9b464.

<https://github.com/hissorii/px68k>

[7] uraraworks, “WebX68k,” main.ts and DESIGN.md, commit b0cd3c7.

<https://github.com/uraraworks/WebX68k/tree/b0cd3c7676d702be5d977a593d40fac825a1139c>

[8] YosAwed, “MPX68K,” GameScene.swift and PX68k core, commit 42fc340.

<https://github.com/YosAwed/MPX68K>